

Freie Universität Berlin

Bachelorarbeit am Institut für Informatik der Freien Universität Berlin

Arbeitsgruppe Cybersecurity and AI

Reduzierung von False Positives in der Cloud-Sicherheit durch hybride Analyse von Metriken und Log-Kontext: Eine skalierbare Architektur-Studie auf CloudAnoBench

Nirosh Heintze

Matrikelnummer: 5585800

niroh04@zedat.fu-berlin.de

Betreuer/in: Hamed H. Fard

Eingereicht bei: Prof. Dr.-Ing. habil. Gerhard Wunder

Berlin, 3. Juni 2026

Zusammenfassung

Automatisierte Anomalie-Erkennung in Cloud-Infrastrukturen erzeugt häufig eine hohe Rate an Fehlalarmen (*False Positives*), die Analysten belasten und die Reaktionsfähigkeit auf echte Sicherheitsvorfälle beeinträchtigen. Diese Arbeit entwickelt und evaluiert eine zweistufige hybride Pipeline auf dem CloudAnoBench-Datensatz: Ein metrischer Anomalie-Detektor (XGBoost auf fünf universellen Systemmetriken) filtert zunächst verdächtige Cases vor; ein log-basierter Validator (TF-IDF-Vektorisierung mit logistischer Regression) verifiziert diese anschließend anhand des Textkontexts der zugehörigen Log-Dateien. Die Evaluation folgt einem leakage-sicheren Ablationsaufbau mit gruppenbasiertem Train-Test-Split.

Die Log-Komponente reduziert die False-Positive-Rate von 79,8 % auf 26,3 % bei einem Recall von 0,958, der die geforderte Mindesterkennungsrate von 0,95 deutlich erfüllt. Ein zentraler empirischer Befund ist die Degeneration des AND-Gates: Der metrische Detektor löst beim gewählten Threshold nahezu alle Test-Cases aus, sodass die AND-Fusion funktional äquivalent zur reinen Log-Filterung wird. Der Log-Kontext erweist sich damit als dominanter Faktor der Pipeline; eine verbesserte Case-Level-Kalibrierung des Metrik-Triggers und eine robustere Evaluation über mehrere Splits bleiben offene Forschungsfragen.

Eidesstattliche Erklärung

Ich versichere hiermit an Eides Statt, dass diese Arbeit von niemand anderem als meiner Person verfasst worden ist. Alle verwendeten Hilfsmittel wie Berichte, Bücher, Internetseiten oder ähnliches sind im Literaturverzeichnis angegeben, Zitate aus fremden Arbeiten sind als solche kenntlich gemacht. Die Arbeit wurde bisher in gleicher oder ähnlicher Form keiner anderen Prüfungskommission vorgelegt und auch nicht veröffentlicht.

3. Juni 2026

Nirosh Heintze

Inhaltsverzeichnis

1	Einleitung	7
1.1	Motivation	7
1.2	Problemstellung	7
1.3	Zielsetzung und Forschungsfrage	7
1.4	Beiträge der Arbeit	8
1.5	Aufbau der Arbeit	9
2	Grundlagen und verwandte Arbeiten	9
2.1	Cloud-Anomalieerkennung und Alert Fatigue	9
2.2	Metrik- und log-basierte Anomalieerkennung	9
2.3	Unbalancierte Klassifikation und Precision-Recall-Metriken	10
2.4	XGBoost für tabellarische Metrikdaten	11
2.5	TF-IDF und logistische Regression für Logdaten	11
2.6	Data Leakage und gruppenbasierte Validierung	11
2.7	Klassifikatorfusion und Case-Level-Aggregation	12
3	Datensatz und Datenaufbereitung	13
3.1	CloudAnoBench und Datenbasis	13
3.2	Datenqualität und Preprocessing	13
3.3	Canonical Feature Set und universelle Features	14
3.4	Leakage-Vermeidung und Split-Strategie	14
4	Metrik-basierte Baseline	15
4.1	Modellwahl und Hyperparameter	15
4.2	Schema-Leakage und Feature-Reduktion	15
4.3	Ergebnisse der Metrik-Baseline	16
5	Log-basierte Verifikation	17
5.1	Log-Daten auf Case-Ebene	17
5.2	TF-IDF-Vektorisierung	17
5.3	Logistische Regression	18
5.4	Ergebnisse der Log-Komponente	19
6	Hybride Fusion und Evaluation	19
6.1	Aggregation auf Case-Ebene	19
6.2	AND-Gate-Fusion und Soft Score	20
6.3	Ablation-Studie	21
6.4	AND-Gate-Degeneration	21
6.5	Qualitätssicherung und Reproduzierbarkeit	22
7	Diskussion und Limitationen	24
7.1	Beantwortung der Forschungsfrage	24
7.2	Rolle des Log-Kontexts	24
7.3	Grenzen der Metrik-Komponente	24
7.4	Methodische Limitationen	25

7.5	Skalierbarkeit als konzeptuelle Einordnung	26
8	Fazit und Ausblick	26
8.1	Zusammenfassung	26
8.2	Beantwortung der Forschungsfrage	26
8.3	Ausblick	27
A	Anhang	30

1 Einleitung

1.1 Motivation

Automatisierte Anomalie-Erkennung ist ein zentrales Werkzeug im Betrieb großer Cloud-Infrastrukturen. Monitoring-Systeme werten kontinuierlich Systemmetriken aus und lösen bei auffälligen Abweichungen Alerts aus. In der Praxis führt dieser Ansatz jedoch zu einem hohen Aufkommen an Fehlalarmen (*False Positives*): Betriebsschwankungen, reguläre Lastspitzen oder geplante Batch-Jobs erzeugen Metrikanomalienmuster, die denen echter Angriffe oder Systemfehler ähneln, ohne es zu sein.

Dieses Phänomen ist in der Literatur als *Alert Fatigue* bekannt und stellt ein anerkanntes operatives Problem im Sicherheitsbetrieb dar. Es beeinträchtigt die Reaktionsfähigkeit von Analysten auf tatsächliche Vorfälle. Die Reduktion der False-Positive-Rate — ohne dabei die Erkennungsrate sicherheitsrelevanter Ereignisse wesentlich zu senken — ist daher eine zentrale Anforderung an praxistaugliche Erkennungssysteme. Abschnitt 2.1 ordnet diesen Befund in die Literatur ein.

1.2 Problemstellung

Ein grundlegendes Problem metrischer Anomalie-Detektoren liegt im Zielkonflikt zwischen Recall und False-Positive-Rate. Um sicherzustellen, dass keine echten Angriffe unentdeckt bleiben (hoher Recall), muss der Entscheidungs-Threshold des Detektors niedrig gesetzt werden. Ein niedriger Threshold markiert jedoch zwangsläufig auch viele tatsächlich unbedenkliche Instanzen als auffällig, was die FPR in die Höhe treibt. Dieser Trade-off ist bei der Verwendung von wenigen, universell verfügbaren Metriken besonders ausgeprägt, da solche Features allein häufig keine ausreichende Trennschärfe zwischen malignen und benignen Fällen bieten.

Log-Dateien liefern einen ergänzenden, qualitativ anderen Informationstyp: Sie enthalten prozessuale Beschreibungen von Systemereignissen, die den Kontext einer Metrik-Auffälligkeit klären können [7]. Eine Lastspitze in der CPU-Auslastung kann sowohl bei einem malignen Prozess als auch bei einem legitimen Backup-Job auftreten — der zugehörige Log-Eintrag hingegen unterscheidet beide Fälle in der Regel klar. Log-Daten eignen sich daher als nachgelagerte Verifikationsstufe: Alerts, die metrisch ausgelöst wurden, können auf Case-Ebene anhand ihres Log-Kontexts gefiltert werden.

Die Kombination beider Signalquellen zu einer hybriden Pipeline — einem metrischen Trigger gefolgt von einer log-basierten Verifikation — ist konzeptuell naheliegend. CloudAnoBench [17] stellt einen synthetischen Benchmark bereit, der für jeden überwachten Case sowohl Metriken als auch Log-Dateien bereitstellt und damit eine kontrollierte Evaluation eines solchen zweistufigen Ansatzes ermöglicht.

1.3 Zielsetzung und Forschungsfrage

Ziel dieser Arbeit ist die Entwicklung und Evaluation einer zweistufigen hybriden Anomalie-Erkennungspipeline auf CloudAnoBench. Der Schwerpunkt liegt auf einer methodisch kontrollierten Evaluation und der expliziten Diskussion der Grenzen des Ansatzes — einschließlich unerwarteter Befunde.

1. Einleitung

Die zentrale Forschungsfrage lautet:

Kann eine zweistufige hybride Pipeline — bestehend aus einem metrischen Anomalie-Detektor und einer log-basierten Nachfilterung — die False-Positive-Rate auf dem CloudAnoBench-Datensatz deutlich senken, ohne die Recall-Rate unter 0,95 zu drücken?

Die Arbeit untersucht diese Frage durch einen kontrollierten Ablations-Aufbau, der eine rein metrische Baseline, eine log-basierte Komponente sowie zwei Fusionsstrategien — ein binäres AND-Gate und eine Soft-Fusion — miteinander vergleicht. Die Evaluation folgt dabei strikten Anforderungen an die Vermeidung von Datenleck und eine leakage-sichere Trainings-Test-Trennung.

Der Begriff *Architektur-Studie* im Titel bezieht sich auf die konzeptuelle Zerlegung der Pipeline in metrischen Trigger, log-basierte Verifikation und Fusionslogik; eine verteilte Implementierung oder messtechnische Skalierungsanalyse ist nicht Gegenstand dieser Arbeit (vgl. Abschnitt 7.5).

1.4 Beiträge der Arbeit

Die vorliegende Arbeit leistet die folgenden konkreten Beiträge:

1. **Reproduzierbare hybride Pipeline auf CloudAnoBench.** Es wird eine in Notebooks dokumentierte, reproduzierbare Pipeline entwickelt, die alle Phasen — Datenaufbereitung, Modelltraining, Fusion und Evaluation — auf CloudAnoBench umsetzt. Alle Ergebnisse sind durch 13 Unit-Tests und `random_state=42` reproduzierbar und nachvollziehbar.
2. **Leakage-bewusste metrische XGBoost-Baseline.** Es wird eine metrische Baseline mit XGBoost [4] entwickelt, die explizit Schema-Leakage durch strukturelle Fehlwerte adressiert. Die Beschränkung auf fünf universell verfügbare Features ist eine methodisch begründete Entscheidung zur Vermeidung von Target Leakage.
3. **Log-basierte Verifikation mit TF-IDF und logistischer Regression.** Es wird eine Case-Ebene-Komponente implementiert, die Log-Dateien mittels TF-IDF vektorisiert und mit logistischer Regression klassifiziert. Die Komponente wird als interpretierbare Baseline — nicht als State-of-the-Art-Ansatz — eingesetzt und gewährleistet durch eine leakage-sichere Trainings-Evaluations-Trennung des Text-Vokabulars eine valide Evaluation.
4. **Ablation der Fusionsstrategien einschließlich Degenerationsbefund.** Es werden zwei Fusionsstrategien — AND-Gate und Soft-Fusion — evaluiert und mit den Einzelkomponenten verglichen. Ein zentraler empirischer Befund ist die Degeneration des AND-Gates: Der metrische Detektor löst beim gewählten Threshold nahezu alle Test-Cases aus, sodass das AND-Gate funktional äquivalent zur reinen Log-Filterung wird. Dieser Befund wird in Abschnitt 6.4 explizit diskutiert.

1.5 Aufbau der Arbeit

Abschnitt 2 legt die konzeptionellen und methodischen Grundlagen der Pipeline dar. Behandelt werden Cloud-Anomalieerkennung, unbalancierte Klassifikation und Precision-Recall-Metriken, XGBoost, TF-IDF, Data Leakage und gruppenbasierte Validierung sowie Klassifikatorfusion und Case-Level-Aggregation.

Abschnitt 3 beschreibt die Pipeline in vier Phasen: Datenaufbereitung und Leakage-Vermeidung, metrische Baseline, log-basierte Verifikation sowie hybride Fusion und Ablation. Die Ergebnisse werden durch Tabellen und Abbildungen dokumentiert; unerwartete Befunde — insbesondere die AND-Gate-Degeneration — werden als eigenständige Abschnitte behandelt.

Abschnitt 4 diskutiert die Ergebnisse im Hinblick auf die Forschungsfrage, bewertet die Rolle der einzelnen Komponenten, benennt die methodischen Limitationen der Arbeit und gibt einen Ausblick auf mögliche Erweiterungen.

2 Grundlagen und verwandte Arbeiten

2.1 Cloud-Anomalieerkennung und Alert Fatigue

Moderne Cloud-Umgebungen werden kontinuierlich über systemseitige Metriken wie CPU-Auslastung, Speicherverbrauch und Netzwerkdurchsatz überwacht. Anomalie-Detektoren werten diese Zeitreihendaten automatisiert aus und generieren Alerts bei auffälligen Abweichungen. In großen, dynamischen Infrastrukturen erzeugen solche Systeme jedoch regelmäßig eine hohe Zahl von Fehlalarmen — sogenannte False Positives — die Sicherheitsanalysten belasten und deren Reaktionsfähigkeit auf echte Vorfälle verringern. Dieser Effekt ist in der Literatur als *Alert Fatigue* bekannt und stellt ein anerkanntes operatives Problem in Security Operations Centres dar [15]. Alahmadi et al. [1] belegen in einer qualitativen Studie, dass in real betriebenen SOCs ein erheblicher Teil der ausgelösten Sicherheitsalarme auf False Positives entfällt, was die Analysekapazität für tatsächliche Vorfälle systematisch einschränkt.

Die False-Positive-Rate (FPR) ist dabei die zentrale operative Zielgröße: Sie beschreibt den Anteil tatsächlich negativer Instanzen, der fälschlicherweise als Alarm eingestuft wird. Eine hohe FPR verursacht nicht nur unnötigen Analyseaufwand, sondern kann im Extremfall dazu führen, dass echte Angriffe im Rauschen untergehen. Die Reduktion der FPR bei gleichzeitiger Aufrechterhaltung einer hohen Erkennungsrate (Recall) bildet daher die Forschungsfrage dieser Arbeit.

Als Evaluierungsgrundlage dient CloudAnoBench [17], ein synthetischer Benchmark, der Systemmetriken und Log-Dateien für Szenarien mit malignem, anomalem und normalem Systemverhalten bereitstellt. CloudAnoBench ist ein neuerer Benchmark und nicht als langjährig etablierter Goldstandard der Community zu verstehen; die in dieser Arbeit erzielten Ergebnisse sind datensatzspezifisch und besitzen keine unmittelbare Allgemeingültigkeit.

2.2 Metrik- und log-basierte Anomalieerkennung

Anomalieerkennung in Cloud-Systemen kann grundsätzlich auf zwei komplementären Informationsquellen operieren: Systemmetriken und Log-Dateien.

Systemmetriken sind numerische Zeitreihendaten, die den Ressourcenzustand eines Systems zu diskreten Zeitpunkten quantifizieren — etwa Prozessorauslastung, Speicherverbrauch oder Netzwerkdurchsatz. Sie sind gut strukturiert, hochfrequent verfügbar und lassen sich effizient mit tabellarischen Lernverfahren verarbeiten. Ihre Schwäche besteht darin, dass unterschiedliche Ereignistypen — etwa legitime Lastspitzen und Angriffsmuster — identische Metrikverläufe erzeugen können. Metriken allein erlauben in solchen Fällen keine zuverlässige Unterscheidung.

Log-Dateien hingegen enthalten textuelle Beschreibungen von Systemereignissen, Prozessaufrufen und Fehlerzuständen. Sie liefern einen prozessualen Kontext, der über die bloße Ressourcensicht hinausgeht. He et al. [7] zeigen in einer umfassenden Übersicht, dass automatisierte Log-Analyse ein etabliertes Werkzeug im Reliability Engineering ist: Textmuster in Log-Dateien können charakteristische Signaturen für unterschiedliche Ereigniskategorien tragen, die in den Metriken nicht sichtbar sind.

Die vorliegende Arbeit verfolgt einen zweistufigen Ansatz: Ein metrischer Detektor identifiziert zunächst potenzielle Auffälligkeiten auf Basis von Ressourcensignalen; eine log-basierte Komponente verifiziert anschließend auf Case-Ebene, ob der Log-Kontext den Alert stützt. Beide Komponenten sind methodisch aufeinander abgestimmt und werden in Abschnitt 3 im Detail beschrieben.

2.3 Unbalancierte Klassifikation und Precision-Recall-Metriken

Anomalie-Erkennungsprobleme sind typischerweise stark unbalanciert: Positive Instanzen (Angriffe, Fehler) sind weit seltener als negative (Normalbetrieb). He und Garcia [6] zeigen, dass in solchen Szenarien standardmäßige Klassifikationsmetriken wie Accuracy irreführend sind, da sie von der Mehrheitsklasse dominiert werden.

Die für diese Arbeit relevanten Metriken sind:

Precision $P = \frac{TP}{TP+FP}$: Anteil der korrekt als positiv klassifizierten Instanzen an allen als positiv gekennzeichneten.

Recall $R = \frac{TP}{TP+FN}$: Anteil der tatsächlich positiven Instanzen, die erkannt werden. In dieser Arbeit als harte Nebenbedingung ($R \geq 0,95$) formuliert.

False-Positive-Rate $FPR = \frac{FP}{FP+TN}$: Anteil der negativen Instanzen, die fälschlicherweise als positiv eingestuft werden. Dies ist die zentrale operative Zielgröße.

Average Precision (AP) Fläche unter der Precision-Recall-Kurve, berechnet als gewichteter Mittelwert der Precision-Werte über alle Recall-Stufen. AP fasst die Klassifikatorleistung über alle möglichen Thresholds zusammen.

Davis und Goadrich [5] zeigen, dass die PR-Kurve gegenüber der ROC-Kurve bei unbalancierten Problemen informativer ist: Während ROC-AUC die Leistung im negativen Regime mitunter überschätzt, macht die PR-Kurve Schwächen bei der Behandlung der Minderheitsklasse direkt sichtbar. Saito und Rehmsmeier [14] bestätigen dies und empfehlen PR-Kurven explizit für stark unbalancierte Klassifikationsprobleme. Diese Befunde motivieren die Verwendung von AUCPR als primäre Optimierungsmetrik in Phase 2 sowie die Fokussierung auf FPR und AP in der Evaluation.

2.4 XGBoost für tabellarische Metrikdaten

Als metrischer Basis-Detektor wird in dieser Arbeit XGBoost [4] eingesetzt. XGBoost implementiert Gradient Tree Boosting und approximiert den Split-Finding-Algorithmus durch eine histogrammbasierte Methode (`tree_method='hist'`), die den Trainingsaufwand gegenüber dem exakten Verfahren deutlich reduziert. Für tabellarisch strukturierte Daten mit numerischen Features und heterogenen Skalen stellt XGBoost ein in der Literatur breit erprobtes Verfahren dar.

Relevante Eigenschaften für diese Arbeit sind der Parameter `scale_pos_weight` zur Gewichtung der Minderheitsklasse im unbalancierten Setting sowie die Möglichkeit, AUCPR direkt als Optimierungsmetrik zu verwenden.

XGBoost wird hier nicht als grundsätzlich überlegenes Verfahren positioniert, sondern als solide, reproduzierbare Baseline für tabellarische Klassifikation. Es ist nicht Ziel dieser Arbeit, das metrische Modell zu optimieren; vielmehr soll seine Leistungsgrenze die Notwendigkeit einer ergänzenden Log-Komponente motivieren.

2.5 TF-IDF und logistische Regression für Logdaten

Die log-basierte Komponente dieser Arbeit nutzt eine Kombination aus *TF-IDF*-Vektorisierung und logistischer Regression. *TF-IDF* (*Term Frequency – Inverse Document Frequency*) gewichtet Terme proportional zu ihrer Häufigkeit im Dokument, normiert jedoch durch ihre Häufigkeit über alle Dokumente. Häufige, wenig diskriminierende Terme — in Log-Dateien etwa routinemäßige Statusmeldungen — erhalten dadurch geringere Gewichte; seltene, charakteristische Terme werden hervorgehoben.

Auf der *TF-IDF*-Matrix wird eine logistische Regression trainiert. Lineare Modelle auf sparse hochdimensionalen Textrepräsentationen sind für diese Problemklasse eine etablierte Wahl; umgesetzt wird der Klassifikator in dieser Arbeit mit `scikit-learn` [12]. Die logistische Regression liefert Wahrscheinlichkeitsschätzungen, die in der nachgelagerten Fusionsstufe direkt weiterverwendet werden.

Die Entscheidung für *TF-IDF* und logistische Regression folgt dem Prinzip einer interpretierbaren und effizient trainierbaren Baseline. Fortgeschrittenere Ansätze zur Log-Analyse — etwa struktur-bewusste Log-Parser, vortrainierte Sprachmodelle oder Embedding-Methoden — wurden in dieser Arbeit nicht untersucht. Sie stellen eine naheliegende Erweiterung dar, sind aber nicht Gegenstand der vorliegenden Evaluation.

2.6 Data Leakage und gruppenbasierte Validierung

Eine korrekte Evaluation maschineller Lernverfahren setzt voraus, dass keine Information aus dem Testset in den Trainingsprozess einfließt. Kaufman et al. [9] klassifizieren verschiedene Formen von Datenleck (*Data Leakage*) und zeigen, dass selbst scheinbar harmlose Preprocessing-Schritte — etwa eine globale Skalierung oder Imputation — die gemessene Testleistung künstlich erhöhen können, wenn sie auf dem Gesamtdatensatz statt ausschließlich auf dem Trainingsanteil durchgeführt werden.

Für diese Arbeit ist eine spezifische Form von Leakage besonders relevant: *Schema-Leakage* durch strukturelle Fehlwerte. Werden verschiedene Klassen mit systematisch unterschiedlichen Feature-Schemata erfasst, entstehen typspezifische NaN-Muster, die

ein Modell implizit zur Bestimmung der Klassenzugehörigkeit nutzen kann — noch bevor inhaltlich relevante Signale verarbeitet werden. Kaufman et al. [9] klassifizieren dies als Target Leakage, da die Feature-Struktur die Zielvariable enkodiert und eine valide Evaluation verhindert. Die methodische Konsequenz ist die Beschränkung der Modellierung auf Features, die in allen Klassen vollständig verfügbar sind.

Ein zweites Leakage-Risiko ergibt sich aus der Datensatzstruktur: Beobachtungen innerhalb desselben Szenarios sind strukturell korreliert. Eine zufällige Aufteilung in Trainings- und Testmenge ohne Berücksichtigung dieser Gruppen würde Szenarien auf beiden Seiten des Splits erlauben und die Trennleistung systematisch überschätzen. Roberts et al. [13] zeigen, dass bei Daten mit gruppenweise korrelierten Beobachtungen eine gruppenweise Partitionierung notwendig ist. In dieser Arbeit wird `GroupShuffleSplit` verwendet, das sicherstellt, dass kein Szenario gleichzeitig in Trainings- und Testmenge vertreten ist.

2.7 Klassifikatorfusion und Case-Level-Aggregation

Die Kombination mehrerer Klassifikatoren zu einem gemeinsamen Entscheidungssystem ist ein etabliertes Forschungsfeld. Kittler et al. [10] systematisieren verschiedene Fusionsregeln — darunter Produkt-Regel, Summen-Regel und Mehrheitsentscheid — und zeigen, unter welchen Voraussetzungen sie asymptotisch äquivalent sind. Das in dieser Arbeit verwendete AND-Gate stellt eine konservative Fusionsstrategie dar: Ein Alert wird nur dann ausgegeben, wenn *beide* Komponenten — metrischer Detektor und Log-Klassifikator — unabhängig voneinander positiv entscheiden. Das AND-Gate maximiert damit die Präzision auf Kosten des Recalls. Ob diese Strategie im konkreten Evaluationsszenario einen Mehrwert gegenüber einer Einzelkomponente liefert, ist eine empirische Frage, die in Abschnitt 6.3 untersucht wird.

Eine eigene methodische Herausforderung ergibt sich aus der unterschiedlichen Granularität beider Komponenten: Der metrische Detektor operiert auf Zeilenebene (einzelne Messpunkte), die Log-Analyse hingegen auf Case-Ebene (eine Log-Datei pro Case). Um beide Ebenen zu vereinen, müssen die zeilenweisen Scores des metrischen Detektors auf Case-Ebene aggregiert werden. In dieser Arbeit wird das Maximum der zeilenweisen Wahrscheinlichkeiten eines Cases verwendet:

$$\hat{p}_{\text{case}} = \max_{i \in \text{case}} \hat{p}_i$$

Diese Max-Aggregation entspricht der Annahme, dass ein einzelner anomaler Messpunkt innerhalb eines Cases als hinreichendes Indiz für eine Auffälligkeit gewertet wird. Konzeptuell ist dieses Prinzip verwandt mit der Instanz-Annahme im *Multiple-Instance Learning* [11]: Ein Bag (hier: Case) gilt als positiv, wenn mindestens eine seiner Instanzen (hier: Zeilen) positiv ist. Die Max-Aggregation ist keine optimierte Fusionsstrategie, sondern eine methodisch begründbare und einfach implementierbare Heuristik.

3 Datensatz und Datenaufbereitung

3.1 CloudAnoBench und Datenbasis

Grundlage der Untersuchung ist CloudAnoBench [17], ein synthetischer Benchmark zur multimodalen Anomalieerkennung in Cloud-Umgebungen. Der Datensatz stellt für jede Beobachtungseinheit sowohl Zeitreihendaten von Systemmetriken als auch zugehörige Log-Dateien bereit und eignet sich damit explizit für eine hybride Analyse, wie sie in dieser Arbeit untersucht wird. Es sei ausdrücklich darauf hingewiesen, dass CloudAnoBench ein neuerer Benchmark ist, der noch nicht als etablierter Goldstandard der Community gilt; die Ergebnisse dieser Arbeit sind daher im Kontext dieses spezifischen Datensatzes zu interpretieren.

Der Datensatz umfasst drei inhaltlich distinkte Typen: *mali* (maliciously anomalous), *anom* (non-malicious anomalous) und *norm* (normal). Die Unterscheidung ist für die binäre Klassifikationsaufgabe relevant, da nur *mali*-Instanzen als positiv (Label 1) und *anom* sowie *norm* als negativ (Label 0) kodiert werden. *Anom*-Fälle stellen dabei nicht-maliziöse technische Anomalien dar, die von einem Sicherheitsdetektor korrekt als unbedenklich eingestuft werden sollen; eine hohe Verwechselungsrate zwischen *mali* und *anom* wäre operativ besonders kostspielig [17]. *Norm*-Cases repräsentieren planmäßigen Betrieb, der aufgrund ressourcenintensiver, aber nicht-maliziöser Aktivitäten — etwa Backup- oder Batch-Prozesse — metrisch auffällig wirken kann. Solche Fälle wurden im Kontext dieser Arbeit als besonders relevante Quelle von Fehlalarmen identifiziert; in der Evaluation werden sie über eine separate FPR_{norm} ausgewertet.

Tabelle 1: Datenbasis: Zeilen und Cases nach Typ

Typ	Zeilen (roh)	Zeilen (clean)	Cases	Label
mali	19 385	19 384	223	positiv (1)
anom	41 400	41 400	460	negativ (0)
norm	51 208	51 208	569	negativ (0)
Gesamt	111 993	111 992	1 252	

Tabelle 1 fasst die Datenbasis zusammen. Lediglich eine einzige Zeile in *mali* wurde beim Bereinigungsverfahren entfernt (0,01 %), was die Datenqualität als insgesamt hoch ausweist. Allen 1 252 Cases ist genau eine Log-Datei zugeordnet; kein Fall weist eine fehlende Log-Datei auf. Der Datensatz deckt insgesamt 45 verschiedene Szenarien ab [17].

3.2 Datenqualität und Preprocessing

Die Rohdaten liegen im CSV-Format vor und weisen zwei typische Qualitätsprobleme auf. Erstens enthalten einige Felder residuale Anführungszeichen aus einer überlappenden CSV-Kodierung (Parameter `quoting=3` beim Einlesen), die durch eine explizite Zeichenkettenbereinigung entfernt werden. Zweitens sind numerische Felder in einigen Dateien als Zeichenketten kodiert und müssen in Fließkommazahlen überführt werden.

3. Datensatz und Datenaufbereitung

Die Median-Imputation fehlender Werte erfolgt in zwei inhaltlich getrennten Kontexten: In Phase 1 wird für die explorative Datenanalyse und die Erstellung der bereinigten Parquet-Artefakte eine Self-Fit-Imputation auf den jeweils vollständig verfügbaren Typ-Daten durchgeführt. Dieser Schritt dient ausschließlich der Datenvorbereitung und Visualisierung; er hat keinen Evaluationsbezug. Für alle evaluationsrelevanten Schritte in Phase 2 und Phase 4 werden die Medianwerte hingegen ausschließlich auf dem Trainingsanteil berechnet, in `data/processed/mali_train_medians.json` gespeichert und danach ohne Neufitting auf die Testdaten angewendet. Diese Trennung schützt vor einem verbreiteten Fehler bei der Vorverarbeitung, der in der Literatur als *data leakage* bezeichnet wird [9].

Die Wahl des Medians gegenüber dem arithmetischen Mittel ist durch die Verteilung der Metriken motiviert: Systemmetriken wie `net_out` zeigen häufig eine rechtsschiefe Verteilung mit einzelnen Ausreißerspitzen, gegen die der Median robust ist [8].

3.3 Canonical Feature Set und universelle Features

Jeder der drei Datensatztypen wird mit einem unterschiedlichen Satz von Systemmetriken erfasst, was auf unterschiedliche Monitoring-Konfigurationen je Datensatztyp hindeutet [17]. Um eine einheitliche Datengrundlage zu schaffen, wird eine *kanonische Feature-Matrix* mit 34 Features aufgebaut, die die Vereinigung aller je gemessenen Metriken darstellt. Fehlende Spalten werden mit NaN aufgefüllt.

Von diesen 34 Features sind lediglich fünf in allen drei Datensatztypen vorhanden:

```
cpu_usage, mem_usage, disk_io, net_in, net_out
```

Die verbleibenden 29 Features sind typspezifisch und strukturell in mindestens einem Typ vollständig als NaN belegt: In mali fehlen 28 von 34 Features strukturell, in anom 23 und in norm lediglich 6. Das Ausmaß dieser strukturellen Fehlwerte ist kein Zufallsartefakt, sondern spiegelt direkt die unterschiedlichen Erhebungsmethoden je Datensatztyp wider. Abbildung 1 zeigt die Verteilung der fünf universellen Features getrennt nach Klasse.

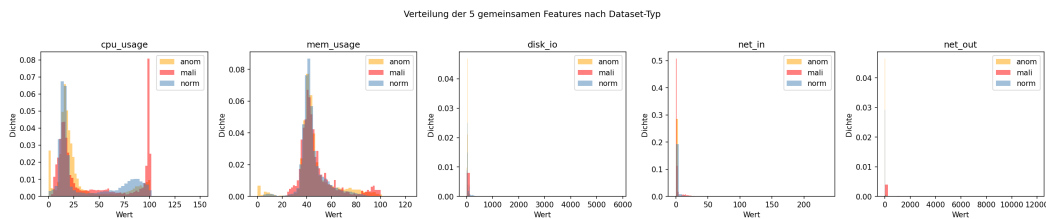


Abbildung 1: Verteilung der fünf universellen Features nach Klasse (positiv = mali, negativ = anom/norm).

3.4 Leakage-Vermeidung und Split-Strategie

Für eine leakage-sichere Evaluation werden Trainings- und Testmenge anhand von Gruppen getrennt, die jeweils die Kombination aus `dataset_type` und `scenario_id` bilden. Die Verwendung von `GroupShuffleSplit` [13] stellt sicher, dass kein Szenario

sowohl in der Trainings- als auch in der Testmenge vertreten ist. Beobachtungen innerhalb desselben Szenarios sind strukturell korreliert; eine zufällige Aufteilung ohne Berücksichtigung dieser Gruppenstruktur würde die Trennleistung im Test systematisch überschätzen.

Der Split erfolgt im Verhältnis 80/20 mit `random_state=42`. Auf Zeilenebene (Phase 2) ergibt sich ein Trainingsset von 89 222 Zeilen und ein Testset von 22 770 Zeilen; auf Case-Ebene (Phase 3 und 4) sind es 999 Trainingsfälle und 253 Testfälle. Das Klassenungleichgewicht beträgt im Trainingsset annähernd 4,92:1 (negativ zu positiv).

4 Metrik-basierte Baseline

4.1 Modellwahl und Hyperparameter

Als Baseline-Modell für den metrikbasierten Anomalie-Detektor wird XGBoost [4] eingesetzt. XGBoost implementiert Gradient Boosting über Entscheidungsbäume und ist durch die `hist`-Methode (histogrammbasiertes Split-Finding) auch für größere Datensätze effizient anwendbar. Für tabellarisch strukturierte Daten mit gemischten Skalen stellt XGBoost eine gut validierte Wahl dar.

Als Optimierungsmetrik wird der Wert der Fläche unter der Precision-Recall-Kurve (*AUCPR*) gewählt. Bei stark unbalancierten Klassifikationsproblemen ist *AUCPR* gegenüber dem ROC-AUC informativer, da Letzterer die Leistung im negativen Regime übergewichtet [5, 14]. Das vorliegende Klassenungleichgewicht von annähernd 4,92:1 erfüllt diese Voraussetzung [6].

Das Klassenungleichgewicht wird in der XGBoost-Verlustfunktion über den Hyperparameter `scale_pos_weight` berücksichtigt. Der Wert von 4,92 wird aus der Trainingsverteilung bestimmt. Als Optimierungsmetrik dient *AUCPR*. Early Stopping ist mit 20 Runden aktiviert. Das beste Modell wurde nach Iteration 109 von 129 beobachteten Iterationen ausgewählt. In dieser prototypischen Einzelsplit-Evaluation wurde Early Stopping auf dem Holdout-Testset überwacht, da kein separater Validierungssplit angelegt wurde. Diese Konfiguration wird in Abschnitt 7.4 als methodische Einschränkung ausgewiesen.

Tabelle 2: XGBoost-Hyperparameter (Phase 2)

Parameter	Wert
<code>tree_method</code>	<code>hist</code>
<code>eval_metric</code>	<code>aucpr</code>
<code>scale_pos_weight</code>	4,92
<code>random_state</code>	42
Early Stopping	aktiv (beste Iteration: 109)

4.2 Schema-Leakage und Feature-Reduktion

Eine direkte Verwendung der vollständigen kanonischen 34-Feature-Matrix würde ein gravierendes Datenleck-Problem erzeugen. Die 29 typspezifischen Features sind strukturell in bestimmten Typen als NaN belegt: Ein Modell, das auf dieser Matrix

trainiert, kann das bloße Muster fehlender Werte zur Bestimmung des Datensatztyps – und damit der Klassenzugehörigkeit – nutzen, noch bevor es irgendein inhaltlichen Signal verarbeitet hat. Kaufman et al. [9] bezeichnen diese Konstellation als *Target Leakage*, da die Feature-Struktur die Zielvariable implizit enkodiert.

Phase 2 wird daher ausschließlich auf den fünf universellen Features trainiert und evaluiert (vgl. Abschnitt 3.3). Diese sind in allen drei Datensatztypen vorhanden und tragen kein typspezifisches Strukturmuster. Die Beschränkung auf diese fünf Features ist eine notwendige Voraussetzung für eine methodisch valide Evaluation; sie schränkt jedoch gleichzeitig die Ausdrucksstärke des Modells ein, was sich in den Ergebnissen widerspiegelt.

4.3 Ergebnisse der Metrik-Baseline

Tabelle 3: Ergebnisse der Metrik-Baseline (XGBoost, Zeilenebene, Testset $n = 22\,770$)

Metrik	Wert
Average Precision (AP)	0,2326
Threshold ($\text{Recall} \geq 0,95$)	0,0222
Recall @ Threshold	0,9500
Precision @ Threshold	0,2180
F1-Maß @ Threshold	0,3546
FPR gesamt	0,7980 (14 724 / 18 450 Negative)
FPR anom	0,8042 (5 501 / 6 840)
FPR norm	0,7944 (9 223 / 11 610)
Inferenz-Latenz	52,1 ms (22 770 Zeilen, indikativ)

Tabelle 3 fasst die Ergebnisse zusammen. Der XGBoost-Detektor erreicht bei einem Threshold von 0,0222 einen Recall von 0,950 und erfüllt damit die harte Nebenbedingung ($\geq 0,95$). Zugleich ist die False-Positive-Rate mit 79,8 % außerordentlich hoch: Von 18 450 negativen Testinstanzen werden 14 724 fälschlicherweise als positiv klassifiziert. Dieser Befund ist auf die Kombination zweier Faktoren zurückzuführen: Erstens stehen lediglich fünf Features zur Verfügung, die für sich genommen eine begrenzte Trennstärke besitzen. Zweitens erzwingt die Recall-Nebenbedingung einen sehr niedrigen Entscheidungs-Threshold von 0,0222, der zwangsläufig einen großen Teil der Negativen als positiv markiert.

Abbildung 2 zeigt die zugehörige Precision-Recall-Kurve. Abbildung 3 visualisiert die Feature-Importances des trainierten Modells nach dem Gain-Kriterium.

Die Metrik-Baseline liefert damit eine klar interpretierbare Ausgangssituation: Nahezu alle malignen Cases werden erkannt, jedoch zum Preis einer sehr hohen Fehlalarmrate von 79,8 %, die den Wert der metrischen Erkennung allein einschränkt. Diese Beobachtung motiviert die in Abschnitt 5 vorgestellte log-basierte Nachfilterung.

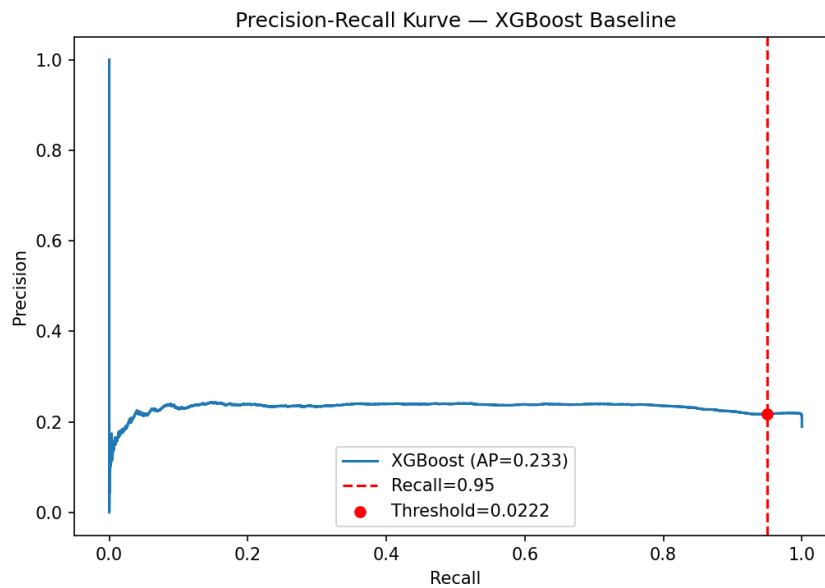


Abbildung 2: Precision-Recall-Kurve der Metrik-Baseline (AP = 0,233). Der markierte Punkt entspricht dem gewählten Threshold 0,0222 bei Recall = 0,950.

5 Log-basierte Verifikation

5.1 Log-Daten auf Case-Ebene

Für jeden der 1 252 Cases liegt eine zugehörige Log-Datei vor. Im Gegensatz zur metrischen Analyse, die auf Zeilenebene (einzelne Messpunkte) operiert, erfolgt die Log-Analyse auf Case-Ebene: Jede Log-Datei wird als ein einzelnes Dokument behandelt, dem genau ein binäres Label zugewiesen ist. Log-Daten enthalten qualitative Textinformationen über Systemereignisse, Fehlermeldungen und Prozessabläufe, die Metriken allein nicht abbilden können. He et al. [7] zeigen in ihrer Übersicht, dass automatisierte Log-Analyse ein etabliertes Werkzeug im Reliability Engineering ist und textbasierte Merkmale relevante Signale für die Anomalieerkennung liefern.

Von 1 252 Cases des Gesamtdatensatzes entfallen nach dem gruppierten Split 999 Fälle auf das Trainingsset und 253 auf das Testset. Im Testset befinden sich 48 positive (mali), 76 anom und 129 norm-Cases.

5.2 TF-IDF-Vektorisierung

Die Textrepräsentation erfolgt mit *Term Frequency – Inverse Document Frequency* (TF-IDF). TF-IDF gewichtet Terme nach ihrer Auftretenshäufigkeit im Dokument, normiert dabei jedoch durch die Häufigkeit über alle Dokumente hinweg, sodass häufige, wenig diskriminierende Terme abgewertet werden. TF-IDF stellt eine interpretierbare und effizient trainierbare Baseline dar; fortgeschrittenere Ansätze wie Log-Parser oder Embedding-Methoden wurden im Rahmen dieser Arbeit nicht untersucht und bilden eine naheliegende Erweiterung. Für Log-Daten ist die TF-IDF-Gewichtung dennoch sinnvoll, da generische Systemereignisse (z. B. regelmäßige Statusmeldungen) häufig

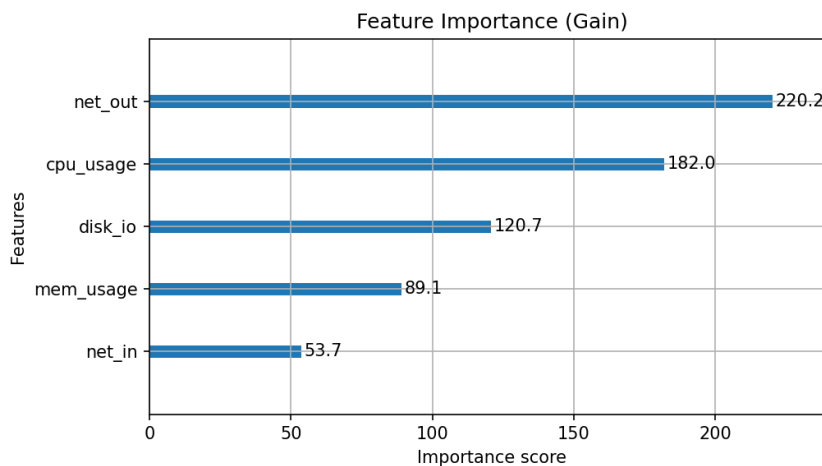


Abbildung 3: Feature-Importances (Gain) des XGBoost-Modells auf den fünf universellen Features.

auftreten, aber geringe Trennkraft besitzen [7].

Tabelle 4: TF-IDF-Konfiguration (Phase 3)

Parameter	Wert
max_features	10 000
ngram_range	(1, 2)
sublinear_tf	True
min_df	2
Fit	ausschließlich auf Trainingsfälle

Der Vektorisierer wird ausschließlich auf den 999 Trainingsfällen gefittet; das Test-Vokabular wird nicht einbezogen [9]. Die Verwendung von Bigrammen erlaubt die Erfassung kurzer Wortfolgen, die in Log-Meldungen häufig als feste Phrasen auftreten; der Parameter `ngram_range` ist auf (1, 2) gesetzt. Mit `sublinear_tf=True` wird die rohe Termfrequenz durch ihren Logarithmus ersetzt, um sehr häufige Terme nicht unverhältnismäßig zu gewichten. Das resultierende Vokabular umfasst 10 000 Terme; die Trainings- und Testmatrix haben die Dimension $(999 \times 10\,000)$ bzw. $(253 \times 10\,000)$.

5.3 Logistische Regression

Auf der TF-IDF-Matrix wird eine logistische Regression trainiert. Lineare Klassifikatoren auf sparse hochdimensionalen Textmatrizen sind für diese Problemklasse eine gut erprobte Wahl; die Implementierung erfolgt über die logistische Regression aus scikit-learn [12]. Der `lbfgs`-Solver ist auf mittelgroßen Datensätzen numerisch stabil und effizient einsetzbar. Das Klassenungleichgewicht (Trainingsverhältnis $\approx 4,71:1$) wird durch `class_weight='balanced'` ausgeglichen, das die Verlustfunktion proportional zu den Klassengrößen gewichtet.

Abbildung 4 visualisiert die 20 einflussreichsten Terme des Modells nach Koeffizientengröße, getrennt nach positiv und negativ gewichteten Termen.

Tabelle 5: Hyperparameter der logistischen Regression (Phase 3)

Parameter	Wert
class_weight	balanced
solver	lbfgs
C	1,0
max_iter	1 000

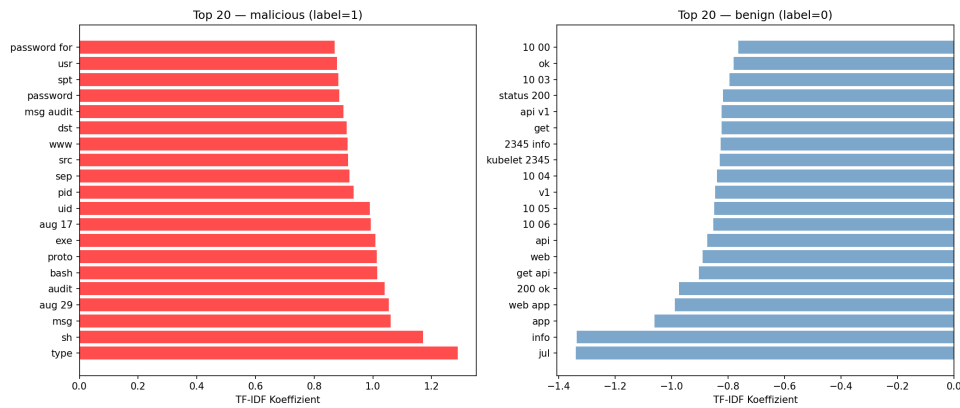


Abbildung 4: Top-20 TF-IDF-Terme nach Koeffizientengröße. Positiv gewichtete Terme sind stärker mit malignen Cases assoziiert; negativ gewichtete Terme treten häufiger in benignen Cases auf.

5.4 Ergebnisse der Log-Komponente

Tabelle 6 zeigt die Ergebnisse der Log-Komponente. Die False-Positive-Rate sinkt gegenüber der Metrik-Baseline von 79,8 % auf 26,3 % – eine Reduktion von mehr als 53 Prozentpunkten – während der Recall von 0,958 die Nebenbedingung klar erfüllt. Abbildung 5 zeigt die zugehörige PR-Kurve.

Ein direkter numerischer Vergleich der Average-Precision-Werte (0,2326 Metrik-Baseline vs. 0,7771 Log-Komponente) ist nur eingeschränkt aussagekräftig. Die Metrik-Baseline wird auf Zeilenebene bewertet (22 770 Testzeilen). Die Log-Komponente wird dagegen auf Case-Ebene mit 253 Testfällen evaluiert. Unterschiedliche Evaluationsgranularitäten führen zu strukturell nicht vergleichbaren PR-Kurven. Aussagekräftig ist hingegen der direkte Vergleich der FPR-Werte auf Case-Ebene (vgl. Abschnitt 6.3).

6 Hybride Fusion und Evaluation

6.1 Aggregation auf Case-Ebene

Die metrische Phase 2 operiert auf Zeilenebene: Für jede Zeile eines Cases wird eine Wahrscheinlichkeit $\hat{p}_{\text{metric}}$ berechnet. Um diesen Zeilenscore auf Case-Ebene zu aggregieren, wird das Maximum über alle Zeilen eines Cases verwendet:

$$\hat{p}_{\text{case}} = \max_{i \in \text{case}} \hat{p}_{\text{metric},i}$$

Tabelle 6: Ergebnisse der Log-Komponente (TF-IDF + LogReg, Case-Ebene, Testset $n = 253$)

Metrik	Wert
Average Precision (AP)	0,7771
Threshold (Recall $\geq 0,95$)	0,1923
Recall @ Threshold	0,9583
Precision @ Threshold	0,4600
F1-Maß @ Threshold	0,6216
FPR gesamt	0,2634 (54 / 205 Negative)
FPR anom	0,2368 (18 / 76)
FPR norm	0,2791 (36 / 129)
Inferenz-Latenz	0,6 ms (253 Cases, indikativ)

Diese Max-Pooling-Operation entspricht der Annahme, dass ein einzelner anomaler Messpunkt innerhalb eines Cases als hinreichendes Indiz für Malicious-Aktivität gewertet werden kann. Dieses Prinzip ist konzeptuell verwandt mit der Aggregationslogik im Multiple-Instance Learning [11], bei dem ein Bag (hier: Case) als positiv gilt, wenn mindestens eine seiner Instanzen (hier: Zeilen) positiv ist. Die Max-Aggregation wird mit einem Threshold von 0,0222 zu einem binären `metric_trigger` konvertiert.

Im Testset von 253 Cases ergibt diese Aggregation folgende Verteilung:

$$\text{metric_trigger} = 1: 252 \text{ Cases} \quad \text{metric_trigger} = 0: 1 \text{ Case}$$

Dieser Befund ist zentral für die Interpretation der nachfolgenden Fusion und wird in Abschnitt 6.4 ausführlich diskutiert.

6.2 AND-Gate-Fusion und Soft Score

Die Fusionsstufe kombiniert den binären Metriktrigger mit der log-basierten Klassifikation zu einem Alert-Signal. Es wurden zwei Fusionsstrategien untersucht:

AND-Gate-Fusion (hart):

$$\text{alert} = \mathbf{1}[\text{metric_trigger} = 1 \wedge \text{log_trigger} = 1]$$

Das AND-Gate setzt einen Alert nur dann, wenn *beide* Komponenten positiv entscheiden. Die konzeptuelle Motivation besteht darin, dass die Log-Komponente als Nachfilter wirkt: Nur Cases, die auch metrisch auffällig sind, werden auf Basis ihres Log-Kontexts abschließend bewertet [10].

Soft-Fusion:

$$s_{\text{fusion}} = \hat{p}_{\text{case}} \times \hat{p}_{\text{log}}$$

Der Soft Score ist das Produkt der metrischen Case-Wahrscheinlichkeit und der log-basierten Wahrscheinlichkeit (vgl. [10]). Da beide Faktoren im Intervall $[0, 1]$ liegen, gilt dies auch für s_{fusion} . Auf Basis dieses Scores kann eine vollständige PR-Kurve berechnet werden.

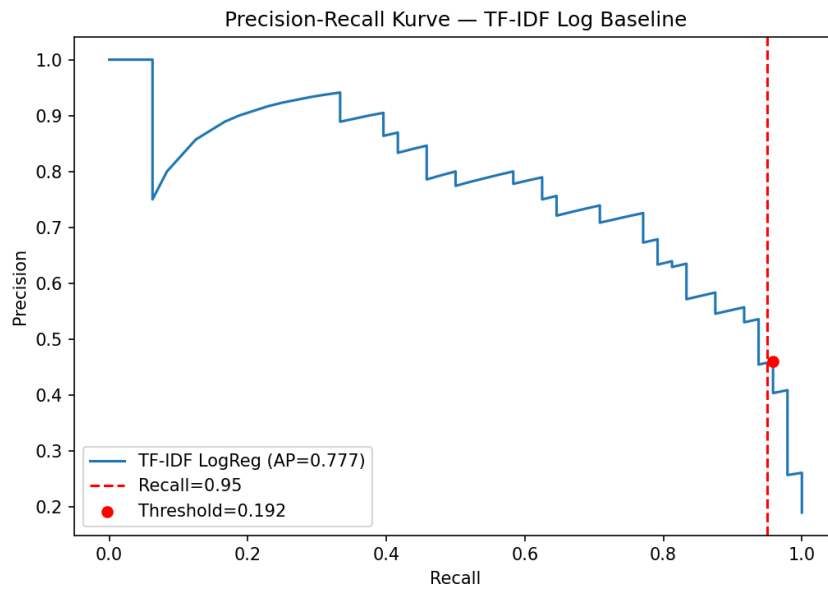


Abbildung 5: Precision-Recall-Kurve der Log-Komponente (AP = 0,777). Der markierte Punkt entspricht dem Threshold 0,192 bei Recall = 0,958.

6.3 Ablation-Studie

Tabelle 7 stellt alle Systemvarianten auf Case-Ebene gegenüber. Abbildung 6 überlagert die PR-Kurven der einzelnen Varianten; Abbildung 7 zeigt den FPR-Vergleich als Balkendiagramm.

Tabelle 7: Ablation-Ergebnisse auf Case-Ebene (Testset: 253 Cases, davon 48 positiv)

Variante	AP*	Recall	Precision	F1	FPR ges.	FPR anom	FPR norm
Metric-only (Zeilen) [†]	0,233	0,950	0,218	0,355	0,798	0,804	0,794
Log-only (Case)	0,777	0,958	0,460	0,622	0,263	0,237	0,279
AND-Fusion	–	0,958	0,460	0,622	0,263	0,237	0,279
Soft-Fusion	0,576	0,958	0,460	0,622	0,263	0,237	0,279

* AP Metric-only auf Zeilenebene; übrige auf Case-Ebene – kein direkter Vergleich.

[†] Zur Referenz; FPR bezieht sich auf 22770 Zeileninstanzen.

Die zentralen Befunde lassen sich wie folgt zusammenfassen: Die Log-Komponente reduziert die FPR von 79,8 % auf 26,3 % (-53,5 Prozentpunkte, -67 % relativ) und erhöht dabei den Recall minimal von 0,950 auf 0,958. AND-Fusion und Soft-Fusion liefern hinsichtlich FPR, Recall, Precision und F1 identische Werte wie Log-only. Das Ranking nach Average Precision ergibt: Log-only (0,777) > Soft-Fusion (0,576).

6.4 AND-Gate-Degeneration

Ein wesentlicher empirischer Befund dieser Arbeit ist die Degeneration des AND-Gates. Von den 253 Test-Cases wird 252 ein `metric_trigger=1` zugewiesen – lediglich ein einziger Case löst den Metrikdetektor nicht aus. Die AND-Fusion wird damit

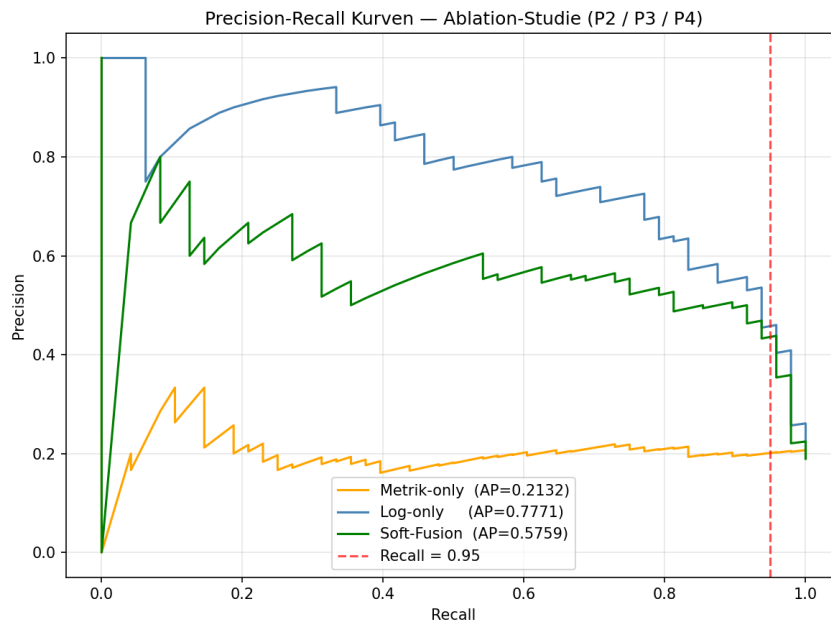


Abbildung 6: Precision-Recall-Kurven der Case-Level-Ablation. Die Metrik-only-Kurve verwendet den case-aggregierten Score $\hat{p}_{\text{case}}$ nach Max-Pooling und ist daher nicht identisch mit der zeilenbasierten AP der Metrik-Baseline aus Tabelle 3. Die Soft-Fusion-Kurve basiert auf dem Produkt-Score $s = \hat{p}_{\text{case}} \times \hat{p}_{\text{log}}$. AP-Werte über unterschiedliche Evaluationsgranularitäten sind nur eingeschränkt direkt vergleichbar.

funktional äquivalent zur reinen Log-Filterung, da die Bedingung `metric_trigger=1` nahezu universell erfüllt ist.

Die Ursache liegt in der erzwungenen Kalibrierung des Metrik-Detektors. Die Recall-Nebenbedingung ($\geq 0,95$) erzwingt bei begrenzter Ausdrucksstärke der fünf universellen Features einen sehr niedrigen Threshold von 0,0222. Ein so niedriger Threshold markiert die überwiegende Mehrheit aller Cases – unabhängig von ihrer tatsächlichen Klasse – als metrisch auffällig. Das AND-Gate, das konzeptionell als Zweistufenfilter gedacht ist, verliert dadurch seine Filterfunktion auf Metrikseite vollständig.

Diese Degeneration ist kein Implementierungsfehler, sondern ein direktes Ergebnis der Eigenschaften des Metrik-Detektors auf dem gewählten Datensatz. Sie schränkt die Aussagekraft der AND-Fusion als eigenständige Fusionsstrategie im vorliegenden Experiment ein und muss bei der Interpretation der Ergebnisse transparent benannt werden. Eine Möglichkeit, die Degeneration zu überwinden, wäre die Anhebung des Metrik-Thresholds auf Kosten des Recalls oder der Einsatz eines ausdrucksstärkeren metrischen Detektors (z. B. mit mehr Features oder anderen Modellklassen). Beides liegt außerhalb des Rahmens dieser Arbeit.

6.5 Qualitätssicherung und Reproduzierbarkeit

Die Kernlogiken der Pipeline werden durch 13 Unit-Tests abgesichert (`test_pipeline.py` im Verzeichnis `tests/`, `pytest`-Framework). Die Tests sind in vier Klassen gegliedert:

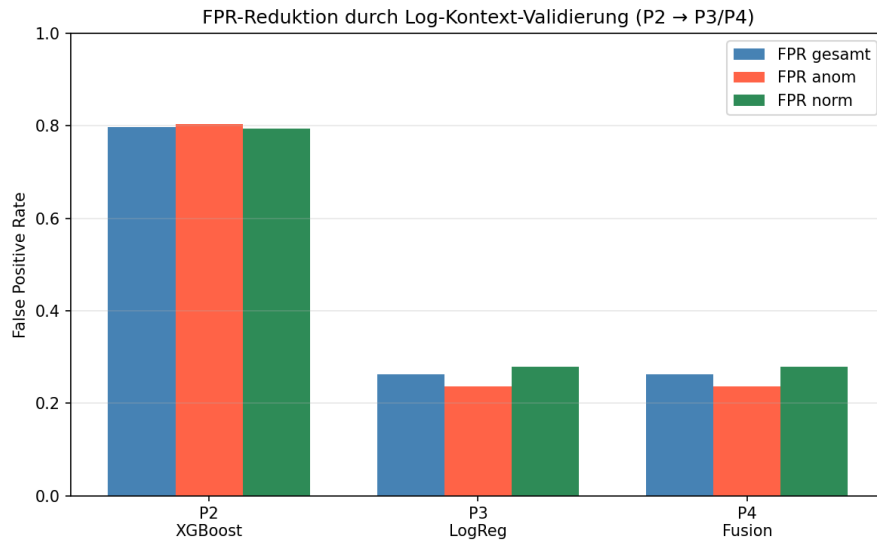


Abbildung 7: Vergleich der False-Positive-Rate zwischen Metrik-Baseline, Log-Komponente und Fusion, aufgeschlüsselt nach Datensatztyp (gesamt, anom, norm).

1. **Canonical Matrix / Preprocessing** (TestBuildCanonicalMatrix, 4 Tests): Prüft die korrekte Behandlung fehlender Spalten, die leakage-sichere Imputation mit Trainings-Medianen, die vollständige NaN-Bereinigung nach Imputation sowie die Koerzion von quotenbehafteten Zeichenketten.
2. **Fusion / Soft Score** (TestFusionLogic, 3 Tests): Verifiziert die AND-Gate-Wahrheitstabelle auf allen vier Eingangskombinationen, die Eigenschaft $FPR_{\text{Fusion}} \leq FPR_{\text{Baseline}}$ auf einem synthetischen Testszenario sowie die Wertebereichseigenschaft des Soft Scores ($s_{\text{fusion}} \in [0, 1]$).
3. **Datenintegrität / GroupShuffleSplit** (TestDataIntegrity, 2 Tests): Stellt sicher, dass keine Gruppe in Train und Test gleichzeitig vorkommt, und prüft das Vorhandensein aller fünf universellen Features in sämtlichen Datensatztypen.
4. **Algorithmische Logik** (TestAlgorithmicLogic, 4 Tests): Prüft die Threshold-Auswahllogik (höchster Threshold bei Recall $\geq 0,95$), die Einhaltung der Recall-Nebenbedingung nach Threshold-Anwendung, die korrekte Max-Aggregation auf Case-Ebene sowie die Abwesenheit testexklusiver Terme im TF-IDF-Vokabular.

Vollständige Reproduzierbarkeit wird durch `random_state=42` in allen Splits und im XGBoost-Training sichergestellt. Die Datei `docs/fusion_cases.csv` enthält alle 253 Test-Cases mit ihren Zwischen-Scores (`max_proba`, `proba_log`, `metric_trigger`, `log_trigger`, `alert`, `score_fusion`) als nachvollziehbarer Audit-Trail.

7 Diskussion und Limitationen

7.1 Beantwortung der Forschungsfrage

Die Forschungsfrage lautete, ob eine zweistufige hybride Pipeline die False-Positive-Rate auf CloudAnoBench deutlich senken kann, ohne die Recall-Rate unter 0,95 zu drücken. Die Ablation-Ergebnisse in Abschnitt 6.3 beantworten diese Frage positiv: Die Log-Komponente reduziert die FPR um 53,5 Prozentpunkte bei einem Recall von 0,958, der die Nebenbedingung klar erfüllt.

Dabei ist das entscheidende Systemelement die Log-Komponente (Phase 3), nicht die AND-Gate-Fusion (Phase 4). Die log-basierte Nachverifikation auf Case-Ebene identifiziert anhand textueller Systemereignismuster, welche Cases trotz metrischer Auffälligkeit nicht als malicious einzustufen sind. Die Fusionsstufe trägt in dieser Konfiguration keinen eigenständigen Beitrag gegenüber der Log-Komponente allein.

7.2 Rolle des Log-Kontexts

Die deutliche FPR-Reduktion durch die Log-Komponente lässt sich durch den zusätzlichen Informationsgehalt der Log-Daten erklären. Log-Meldungen enthalten semantisch reichhaltige Ereignisbeschreibungen. Diese können Ursache und Art eines Betriebsvorgangs qualitativ charakterisieren. Metriken hingegen messen quantitative Systemzustände und können zwischen unterschiedlichen Ereignistypen mit identischer Intensität nicht unterscheiden. TF-IDF kann diskriminierende Terme identifizieren, die in malignen Cases systematisch häufiger oder seltener auftreten als in normalen oder anomalen Cases [7].

Die Analyse auf Case-Ebene statt Zeilenebene ist dabei konzeptuell angemessen: Die Entscheidung, ob ein Security-Alert ausgelöst werden soll, bezieht sich auf eine Beobachtungseinheit (Case) als Ganzes, nicht auf einen einzelnen Messpunkt.

7.3 Grenzen der Metrik-Komponente

Der metrische Detektor erzielt mit einer AP von 0,233 eine verhältnismäßig schwache Diskriminationsleistung. Dies ist auf zwei Faktoren zurückzuführen. Erstens beschränkt die Schema-Leakage-Vermeidung (vgl. Abschnitt 3.3) das Modell auf fünf Features, die zwar universell verfügbar, aber in ihrer Kombination nicht hochdiskriminativ für mali gegenüber anom sind. Zweitens erlaubt die Zeilenebene keine Kontextualisierung einzelner Messpunkte: Ein kurzzeitiger Anstieg von `cpu_usage` kann sowohl bei einem malignen Vorgang als auch bei normaler Lastspitze auftreten.

Die Konsequenz ist ein sehr niedriger operativer Threshold (0,0222), der zur AND-Gate-Degeneration und zur hohen FPR führt. Ein metrischer Detektor mit höherer Ausdrucksstärke – etwa durch Einbeziehung von Zeitfenster-Aggregaten, typspezifischen Features oder tieferen Modellen – könnte diese Einschränkungen möglicherweise mildern, ist jedoch mit dem Risiko erhöhten Schema-Leakages verbunden.

7.4 Methodische Limitationen

Mehrere methodische Einschränkungen begrenzen die Generalisierbarkeit der Befunde und müssen transparent benannt werden.

Einzelner Holdout-Split. Es wurde lediglich ein einziger GroupShuffleSplit mit `n_splits=1` durchgeführt. Die Streuung der Ergebnismetriken über verschiedene Splits ist damit unbekannt. Cawley und Talbot [3] zeigen, dass Designentscheidungen, die am Testset orientiert werden, zu Optimismus-Verzerrung führen können. Da Hyperparameter und Threshold in dieser Arbeit auf Basis des einzigen Testsets gewählt wurden, ist ein solcher Bias nicht vollständig ausschließbar.

Kleine positive Testmenge. Das Testset enthält lediglich 48 positive Cases (mali). Konfidenzintervalle für die berichteten Metriken wurden nicht berechnet; die Punktschätzungen der FPR und des Recalls sind daher mit entsprechender Vorsicht zu interpretieren.

Synthetischer Datensatz. CloudAnoBench ist ein synthetisch generierter Datensatz und kein aufgezeichneter Produktions-Traffic. Ob die gelernten Textmuster auf andere Cloud-Umgebungen, andere Log-Formate oder reale Angriffe übertragbar sind, ist durch diese Arbeit nicht untersucht. Arp et al. [2] weisen allgemein darauf hin, dass die Übertragbarkeit von im Labor evaluierten ML-Sicherheitsmodellen auf reale Einsatzumgebungen systematisch überschätzt werden kann.

Unterschiedliche Evaluationsgranularitäten. Metrik-Baseline und Log-Komponente werden auf unterschiedlichen Ebenen evaluiert (Zeilen vs. Cases), was einen direkten numerischen Vergleich der AP-Werte ausschließt. Nur auf Case-Ebene sind die FPR-Werte direkt vergleichbar.

Early-Stopping-Konfiguration. Das Early Stopping der Metrik-Baseline wurde auf dem Holdout-Testset überwacht; ein separater Validierungssplit wurde nicht angelegt. Dadurch können die zeilenbasierten Metrik-Ergebnisse leicht optimistisch sein [3]. Da die Metrik-Baseline trotz dieser Konfiguration die schwächere Komponente bleibt und der Operating-Threshold post-hoc unter der Recall-Nebenbedingung gewählt wird, ändert dies nicht die qualitative Interpretation der AND-Gate-Degeneration. Für eine belastbarere Evaluation wären ein separater Validierungssplit oder verschachtelte Validierung erforderlich.

Soft-Fusion ohne Mehrwert gegenüber Log-only. Die Soft-Fusion-Strategie ($s = \hat{p}_{\text{case}} \times \hat{p}_{\text{log}}$) erzielt eine AP von 0,5759, die unter der AP der Log-Komponente allein (0,7771) liegt. Die Multiplikation mit dem metrischen Faktor dämpft den Log-Score systematisch, ohne einen eigenständigen Informationsgewinn zu liefern. Tax et al. [16] zeigen, dass die Produktregel bei abhängigen oder redundanten Repräsentationen gegenüber einfachen Mittelungsstrategien schlechter abschneiden kann. Eine alternative Fusionsgewichtung oder ein kalibrierter metrischer Detektor könnten diesen Nachteil möglicherweise beheben, wurden aber nicht untersucht.

Prototypischer Implementierungscharakter. Die gesamte Pipeline ist in Jupyter Notebooks implementiert und als Forschungsprototyp zu verstehen. Es existiert keine paketierte, produktionsreife Software-Implementierung.

Keine Einzelfallanalyse der norm-Cases. Die Evaluation weist die FPR für norm-Cases (planmäßiger Betrieb, darunter Backup-Vorgänge und Batch-Jobs) separat aus ($\text{FPR}_{\text{norm}} = 27,9\%$, 36 von 129 Cases). Eine qualitative Analyse der individuellen falsch

positiv klassifizierten norm-Cases — etwa hinsichtlich gemeinsamer Log-Muster oder Szenariotypen — wurde nicht durchgeführt. Die Zwischenergebnisse aller 253 Test-Cases, einschließlich metrischer und log-basierter Scores, sind in `docs/fusion_cases.csv` dokumentiert und stehen für Anschlussanalysen bereit.

7.5 Skalierbarkeit als konzeptuelle Einordnung

Konzeptuell ließe sich die Pipeline als verteilbare Abfolge von Filter- und Verifikationsschritten auffassen, etwa in Form einer Map/Reduce-artigen Aufteilung: Die metrische Vorab-Filterung (Phase 2) lässt sich als parallele Map-Operation auf Zeilenebene interpretieren, die log-basierte Verifikation (Phase 3) als fallweise Aggregation (Reduce).

Es ist jedoch ausdrücklich festzuhalten, dass diese Skalierbarkeitsbetrachtung konzeptueller Natur ist. Eine verteilte Implementierung wurde im Rahmen dieser Arbeit weder realisiert noch gemessen. Die berichteten Latenzzeiten (Phase 2: 54,7 ms, Phase 3: 2,4 ms, gesamt sequenziell: 57,1 ms) beziehen sich auf die sequenzielle Ausführung des Fusion-Notebooks auf einem einzelnen Rechner; eine detaillierte Hardware-Spezifikation liegt nicht vor. Die in den Einzelnotizbüchern gemessenen Werte (Phase 2: 52,1 ms, Phase 3: 0,6 ms) können aufgrund abweichender Ausführungskontexte und Laufzeitumgebungen leicht davon abweichen; alle Latenzangaben sind daher als indikative Messungen zu verstehen.

Eine Einbettung der Pipeline in Data-Intelligence-Plattformen wie Databricks oder Snowflake sowie ein automatisiertes Modell-Lifecycle-Management — etwa über MLflow zur Überwachung von Model Drift in dynamischen Cloud-Umgebungen — wären naheliegende Erweiterungen für eine produktionsnahe Umsetzung. Diese Aspekte wurden in der vorliegenden Arbeit weder implementiert noch empirisch evaluiert und liegen außerhalb des Scopes dieser Benchmark-Studie. Die Skalierbarkeitsbetrachtung bleibt daher auf die konzeptuelle Zerlegung der Pipeline in verteilbare Filter- und Verifikationsschritte beschränkt.

8 Fazit und Ausblick

8.1 Zusammenfassung

Diese Arbeit hat eine zweistufige hybride Pipeline zur Reduzierung von False Positives in der Cloud-Anomalieerkennung entwickelt und auf dem synthetischen Benchmark CloudAnoBench evaluiert. Der methodische Fokus lag auf einer leakage-sicheren Evaluation unter einer festen Recall-Nebenbedingung ($R \geq 0,95$), nicht auf der Maximierung einzelner Metriken. Beide Komponenten — metrischer Detektor und Log-Verifikation — wurden isoliert und in zwei Fusionskonfigurationen evaluiert; alle Ergebnisse sind durch einen festen Zufallszustand und 13 Unit-Tests reproduzierbar.

8.2 Beantwortung der Forschungsfrage

Die Forschungsfrage wird bejaht: Die Log-Komponente reduziert die FPR von 79,8 % auf 26,3 % bei einem Recall von 0,958 — die Recall-Nebenbedingung ist damit erfüllt.

Die vollständige Ergebnisanalyse und Argumentation findet sich in Abschnitt 6.3 und den Abschnitten zur Diskussion.

Der qualifizierende Befund ist, dass Log-Kontext auf Case-Ebene als eigenständiger Prädiktor dominiert und die Fusionsstufe keinen zusätzlichen Beitrag liefert: Das AND-Gate verliert aufgrund des niedrigen Metrik-Thresholds seine Filterfunktion auf Metrikseite; die Soft-Fusion erzielt eine niedrigere AP als die Log-Komponente allein. Die Nützlichkeit einer AND-Fusion hängt damit entscheidend von der Kalibrierung des Metrik-Triggers auf Case-Ebene ab, die in dieser Arbeit nicht umgesetzt wurde.

Diese Befunde gelten für den CloudAnoBench-Datensatz unter den gewählten Konfigurationen und sind ohne weitere Evidenz nicht zu verallgemeinern.

8.3 Ausblick

Aus den Befunden dieser Arbeit ergeben sich mehrere konkrete Ansatzpunkte für zukünftige Arbeiten:

Case-Level-Kalibrierung des Metrik-Triggers. Die AND-Gate-Degeneration ist eine direkte Folge der Threshold-Wahl auf Zeilenebene. Eine Kalibrierung direkt auf aggregierten Case-Scores — oder der Einsatz alternativer Aggregationsregeln wie Median statt Maximum — würde die AND-Gate-Logik erst operativ wirksam machen und den eigenständigen Beitrag der Metrik-Komponente zur Fusion prüfbar machen.

Alternative Fusionsregeln. Neben AND-Gate und Produkt-Regel existieren weitere Fusionsstrategien — etwa Summen-Regel, gewichtete Mehrheitsentscheidung oder lernbasierte Kombinatoren. Ob solche Regeln auf CloudAnoBench einen messbaren Vorteil gegenüber der Log-Komponente allein erzielen, bleibt eine offene Frage.

Weiterentwicklung der Log-Komponente. Die log-basierte Verifikation wurde in dieser Arbeit als interpretierbare Baseline mit TF-IDF und logistischer Regression umgesetzt. Ob struktur-bewusste Log-Parser, vortrainierte Sprachmodelle oder Embedding-Methoden auf diesem Datensatz zu einer weiteren FPR-Reduktion führen, wäre eine naheliegende Erweiterung.

Statistische Absicherung. Die berichteten Metriken basieren auf einem einzigen Trainings-Test-Split. Die Frage, wie stabil die Ergebnisse über verschiedene Splits sind, ließe sich durch Bootstrap-Konfidenzintervalle oder wiederholte Splits mit unterschiedlichen Zufallszuständen beantworten.

Evaluation auf realem Produktions-Traffic. CloudAnoBench verwendet kontrollierte Anomalie-Injektionen. Ob die gelernten Muster auf reale Infrastrukturdaten — mit natürlichen Klassenverteilungen, variablen Log-Formaten und unbekannten Anomalietypen — übertragbar sind, ist durch diese Arbeit nicht untersucht und stellt eine wichtige Anschlussfrage dar.

Modularisierung der Pipeline. Die funktionale Pipeline liegt derzeit überwiegend in Jupyter Notebooks. Eine Modularisierung in eine paketierbare, konfigurierbare

Software-Pipeline würde die Wiederverwendbarkeit und Anpassbarkeit an andere Datensätze wesentlich verbessern.

Literatur

- [1] Bushra A. Alahmadi, Louise Axon und Ivan Martinovic. „99% False Positives: A Qualitative Study of SOC Analysts’ Perspectives on Security Alarms“. In: *31st USENIX Security Symposium (USENIX Security 22)*. 2022, S. 2783–2800.
- [2] Daniel Arp u. a. „Dos and Don’ts of Machine Learning in Computer Security“. In: *31st USENIX Security Symposium (USENIX Security 22)*. 2022, S. 3971–3988.
- [3] Gavin C. Cawley und Nicola L. C. Talbot. „On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation“. In: *Journal of Machine Learning Research* 11 (2010), S. 2079–2107.
- [4] Tianqi Chen und Carlos Guestrin. „XGBoost: A Scalable Tree Boosting System“. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016, S. 785–794. DOI: 10.1145/2939672.2939785.
- [5] Jesse Davis und Mark Goadrich. „The Relationship Between Precision-Recall and ROC Curves“. In: *Proceedings of the 23rd International Conference on Machine Learning (ICML)*. ACM, 2006, S. 233–240. DOI: 10.1145/1143844.1143874.
- [6] Haibo He und Edwardo A. Garcia. „Learning from Imbalanced Data“. In: *IEEE Transactions on Knowledge and Data Engineering* 21.9 (2009), S. 1263–1284. DOI: 10.1109/TKDE.2008.239.
- [7] Shilin He u. a. „A Survey on Automated Log Analysis for Reliability Engineering“. In: *ACM Computing Surveys* 54.6 (2022), 130:1–130:37. DOI: 10.1145/3460345.
- [8] Peter J. Huber und Elvezio M. Ronchetti. *Robust Statistics*. 2. Aufl. Wiley, 2009. DOI: 10.1002/9780470434697.
- [9] Shachar Kaufman u. a. „Leakage in Data Mining: Formulation, Detection, and Avoidance“. In: *ACM Transactions on Knowledge Discovery from Data* 6.4 (2012), 15:1–15:21. DOI: 10.1145/2382577.2382579.
- [10] Josef Kittler u. a. „On Combining Classifiers“. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 20.3 (1998), S. 226–239. DOI: 10.1109/34.667881.
- [11] Oded Maron und Tomás Lozano-Pérez. „A Framework for Multiple-Instance Learning“. In: *Advances in Neural Information Processing Systems 10 (NIPS 1997)*. MIT Press, 1998, S. 570–576.
- [12] F. Pedregosa u. a. „Scikit-learn: Machine Learning in Python“. In: *Journal of Machine Learning Research* 12 (2011), S. 2825–2830.
- [13] David R. Roberts u. a. „Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure“. In: *Ecography* 40.8 (2017), S. 913–929. DOI: 10.1111/ecog.02881.

- [14] Takaya Saito und Marc Rehmsmeier. „The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets“. In: *PLOS ONE* 10.3 (2015), e0118432. DOI: 10.1371/journal.pone.0118432.
- [15] Shahroz Tariq u. a. „Alert Fatigue in Security Operations Centres: Research Challenges and Opportunities“. In: *ACM Computing Surveys* 57.9 (Apr. 2025), S. 1–38. DOI: 10.1145/3723158.
- [16] David M. J. Tax u. a. „Combining multiple classifiers by averaging or by multiplying?“ In: *Pattern Recognition* 33.9 (2000), S. 1475–1485. DOI: 10.1016/S0031-3203(99)00138-7.
- [17] Jinyu Zou u. a. *Towards Generalizable Context-aware Anomaly Detection: A Large-scale Benchmark in Cloud Environments*. arXiv:2508.01844. 2025.

A Anhang

Die zur Reproduktion relevanten Artefakte befinden sich im Repository: die vier Jupyter-Notebooks (01_data_exploration, 02_metric_baseline, 03_log_component, 04_fusion), die erzeugten Abbildungen im Verzeichnis docs/ sowie die 13 Unit-Tests in tests/test_pipeline.py.

Ergänzend liegt im Repository ein Dockerfile vor, das eine einfache Jupyter-Lab-Umgebung auf Basis von Python 3.12 mit den benötigten Abhängigkeiten bereitstellt. Es dient der lokalen Reproduzierbarkeit der Notebook-Ausführung und ist nicht als produktionsreife Deployment-Umgebung zu verstehen. Der Container kann aus dem Repository-Wurzelverzeichnis mit `docker build -t cloudanobench-thesis .` gebaut und anschließend mit `docker run -p 8888:8888 cloudanobench-thesis` gestartet werden.